

Τμήμα Μηχανικών Η/Υ & Πληροφορικής
Πανεπιστήμιο Πατρών

Εισαγωγή στον Προγραμματισμό Υπολογιστών

Η γλώσσα προγραμματισμού C

Συνοπτική περιγραφή διαδικασίας εκτέλεσης Δραστηριότητας

1	Γραφική αναπαράσταση της Διεργασίας
2	Λεκτική Περιγραφή
3	Μετατροπή Λεκτικής Περιγραφής σε Πηγαίο Κώδικα

Η διεργασία παρασκευής καφέ – Συνθετικά

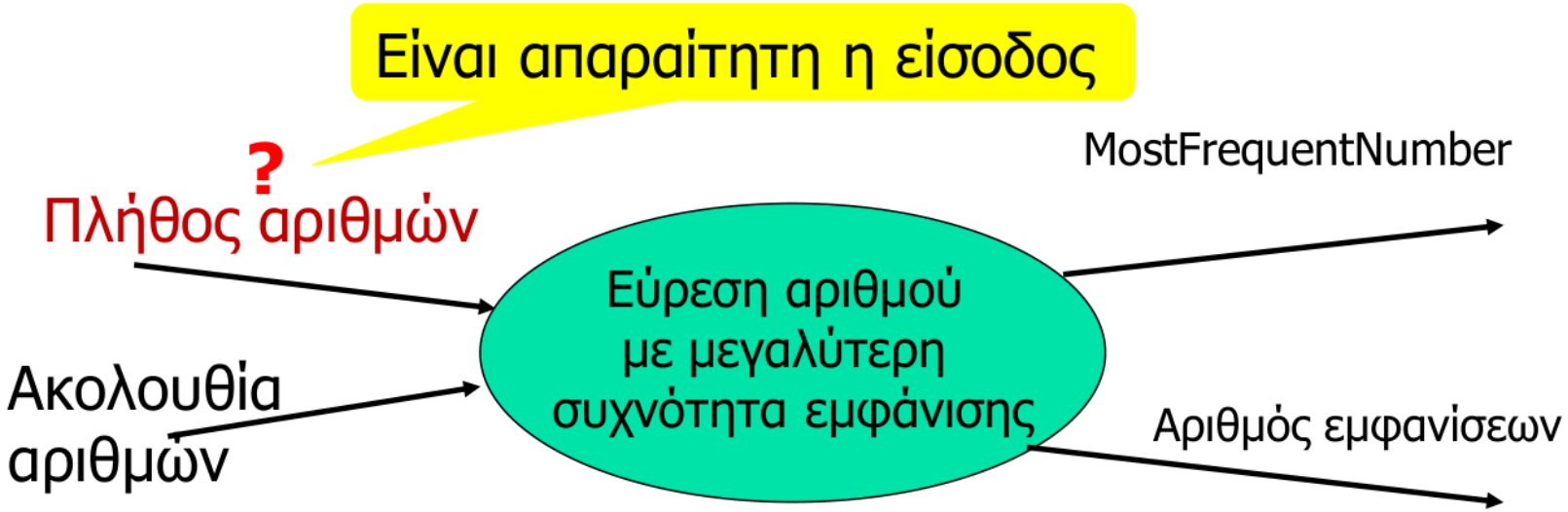


- Η διεργασία έχει :
  - **Όνομα** (Ορίζει το **τι κάνει** η διεργασία)
  - **Περιγραφή** (Ορίζει το **πως το κάνει**)
  - Δέχεται **εισόδους** (τις οποίες επεξεργάζεται)
  - Παράγει **έξοδο/δους** (τις οποίες διαμορφώνει με βάση τις εισόδους)

MostFrequentNumber – Ενέργειες Ε1, Ε2

Συχνότερα εμφανιζόμενος αριθμός

- Αναπτύξτε ένα πρόγραμμα σύμφωνα με το οποίο το σύστημα θα διαβάζει μία ακολουθία από n αριθμούς και θα εμφανίζει: α) τον αριθμό που εμφανίστηκε στην ακολουθία περισσότερες συνεχόμενες φορές και β) το πλήθος των συνεχόμενων εμφανίσεων του.



MostFrequentNumber–Ε3(Λεκτική Περιγραφή) 1/3

12 12 91 62 62 62 120 120 57 ....

12 αριθμός

12 Υποψήφιος Συχνότερος Αριθμός (ΥΣΑ)

1 Πλήθος Εμφανίσεων (ΠΕ) ΥΣΑ

Αναγνωσιμότητα (Readability)
Απλότητα (Simplicity)

Πάρε τον πρώτο αριθμό

Θεώρησε τον αριθμό ΥΣΑ με ΠΕ 1

Για κάθε επόμενο αριθμό της ακολουθίας

- **Αν** αυτός είναι ίσος με τον ΥΣΑ
  - Αύξησε κατά 1 το ΠΕΥΣΑ
- **Αλλιώς**
  - Αντικατέστησε τον ΥΣΑ με τον αριθμό

Συνεχίστε τον έλεγχο για διάφορα σενάρια ακολουθίας

Εμφάνισε Συχνότερο Αριθμό και Πλήθος Εμφανίσεων

MostFrequentNumber–Ε3(Λεκτική Περιγραφή) 2/3

62	αριθμός	12 12 91 62 62 62 120 120 57 ….	
62	Υποψήφιος Συχνότερος Αριθμός	3	Πλήθος Εμφανίσεων ΥΣΑ
12	Συχνότερος Αριθμός	2	Πλήθος Εμφανίσεων ΣΑ

….

Για κάθε επόμενο αριθμό της ακολουθίας

- **Αν** αυτός είναι ίσος με τον ΥΣΑ
  - Αύξησε κατά 1 το ΠΕΥΣΑ
  - **Αν** ο ΠΕΥΣΑ είναι μεγαλύτερος από το ΠΕΣΑ
    - Αντικατέστησε τον ΣΑ με τον ΥΣΑ
    - Αντικατέστησε το ΠΕΣΑ με ΠΕΥΣΑ
- **Αλλιώς**
  - Αντικατέστησε τον ΥΣΑ με τον αριθμό

Εμφάνισε ΣΑ και ΠΕΣΑ

Βρείτε λάθη
Βελτιώστε τον

MostFrequentNumber–Ε3(Λεκτική Περιγραφή) 3/3

12 12 91 62 62 62 120 120 57 ….

Πάρε τον πρώτο αριθμό

Θεώρησε τον αριθμό ΥΣΑ με ΠΕ 1

Για κάθε αριθμό της ακολουθίας

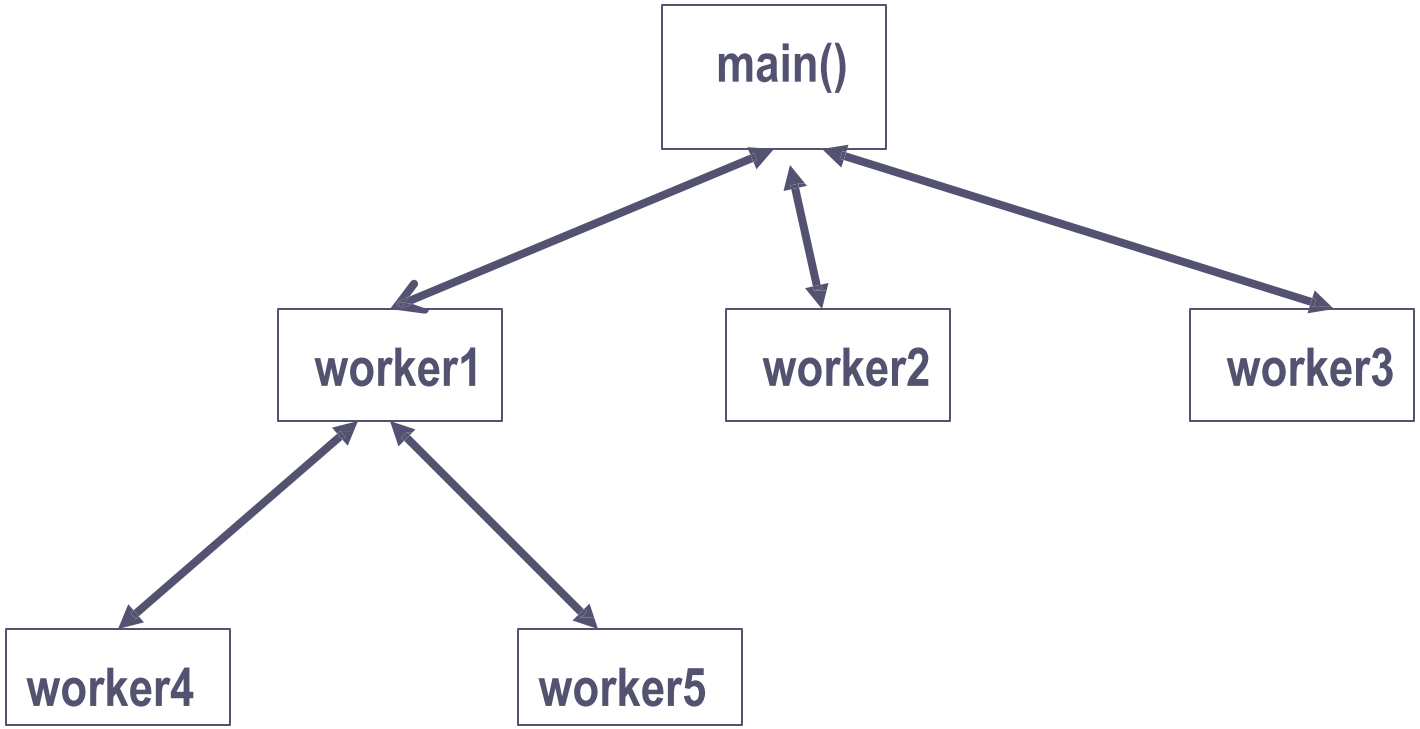
- **Αν** αυτός είναι ίσος με τον ΥΣΑ
  - Αύξησε κατά 1 το ΠΕΥΣΑ
  - **Αν** ο ΠΕΥΣΑ είναι μεγαλύτερος από το ΠΕΣΑ
    - Αντικατέστησε τον ΣΑ με τον ΥΣΑ
    - Αντικατέστησε το ΠΕΣΑ με ΠΕΥΣΑ
- **Αλλιώς**
  - Αντικατέστησε τον ΥΣΑ με τον αριθμό

Εμφάνισε ΣΑ και ΠΕΣΑ

Αναγνωσιμότητα (Readability)
Απλότητα (Simplicity)

Συνεχίστε τον έλεγχο για διάφορα σενάρια Ακολουθίας. Βρείτε λάθη

Συναρτήσεις (Functions)



Υποπρογράμματα

- Αποτελούν τον τρόπο εφαρμογής (υλοποίησης) της τμηματοποίησης σε ένα πρόγραμμα.
- Κάθε υποπρόγραμμα είναι ένα αυτόνομο τμήμα (μικρό πρόγραμμα)
- Ο συνδυασμός των υποπρογραμμάτων συνιστά το (αρχικό) πρόγραμμα.
- Πλεονεκτήματα
  - Αποφυγή επαναλήψεων
  - Αύξηση επαναχρησιμοποίησης
  - Βελτίωση αναγνωσιμότητας
  - Ευκολότερη συντήρηση

Συναρτήσεις στη C

- Πρόγραμμα C = σύνολο συναρτήσεων
- Η συνάρτηση main αντιπροσωπεύει το κυρίως πρόγραμμα, δηλ. τον τρόπο με τον οποίο συνδυάζονται οι υπόλοιπες συναρτήσεις για τη λύση του προβλήματος.
- Μέρη συνάρτησης
  - Κεφαλίδα = η διεπαφή της συνάρτησης (όνομα, είσοδος, έξοδος)
  - Σώμα = υλοποίηση/ορισμός της συνάρτησης
- Προτάσεις συνάρτησης
  - Δήλωση – Κλήση - Ορισμός

Δήλωση Συνάρτησης

- Προσδιορίζεται η διεπαφή, ο τρόπος αναφοράς στη συνάρτηση:

  <τύπος> <όνομα – συν> ([<παράμετροι>]); Όπου

  <παράμετροι>:=<τύπος1> [<όνομ-παρ1>], …, <τύποςN> [<όνομ-παρN]

  τυπικά ορίσματα (ή είσοδοι)

  τυπικά αποτελέσματος (εξόδου)

- Αν η συνάρτηση δεν επιστρέφει κάποια τιμή, τότε χρησιμοποιείται σαν τύπος αποτελέσματος η λέξη κλειδί void.

Παραδείγματα

```
int max (int a, int b);

int min (int, int);

double exp ( double m, int n);

void swap (int a, int b);

char *getptr (char * str, char ch);

char *getptr (char str [ ], char ch);
```

Παράδειγμα

```
#include <stdio.h>
int square(int y); /* αρχέτυπο συνάρτησης */
int main ( )
{
    int x;
    for (x=1; x<=10; x++)
            printf("%d", square(x));
    printf("\n");
    return 0;
}
/* ορισμός συνάρτησης */
int square( int y)
{
    return y * y;
}
```

Κλήση Συνάρτησης

- Καλείται η συνάρτηση για εκτέλεση με συγκεκριμένα ορίσματα

  <όνομα – συν> (<ορισ1>, <ορισ2>, …, <ορισN>);

  πραγματικά ορίσματα (σταθερές, μεταβλητές, εκφράσεις|)

- Τα πραγματικά ορίσματα πρέπει να είναι του ίδιου αριθμού και τύπου με τα τυπικά ορίσματα

Παραδείγματα

```
swap ( x, y);

draw_circle (a/2.0, 2.0 * b, c);

max_num = max(num1, num2);

min_num = (num, 5);

x = y + max (num1/2.0, 3.0*num2);

printf ( “ο μέγιστος είναι: %d\n”, max(x1,x2));
```

Ορισμός Συνάρτησης

- Για κάθε μη ενσωματωμένη (δική μας) συνάρτηση πρέπει να ορίσουμε το σώμα της στο πρόγραμμα (μετά την main)

```
<τύπος> (<όνομα - συν> (<παράμετροι>)
        {
                <δηλώσεις τοπικών μεταβλητών>

                <προτάσεις>

        }
```

Παράδειγμα (1)

```
double embadon (double platos, double mikos)
{
    double apotelesma; apotelesma = platos * mikos;  platos=15;

    return (apotelesma);
}
```

τυπικές παράμετροι

τοπική μεταβλητή

Επιστροφή ελέγχου και τιμής στην καλούσα συνάρτηση

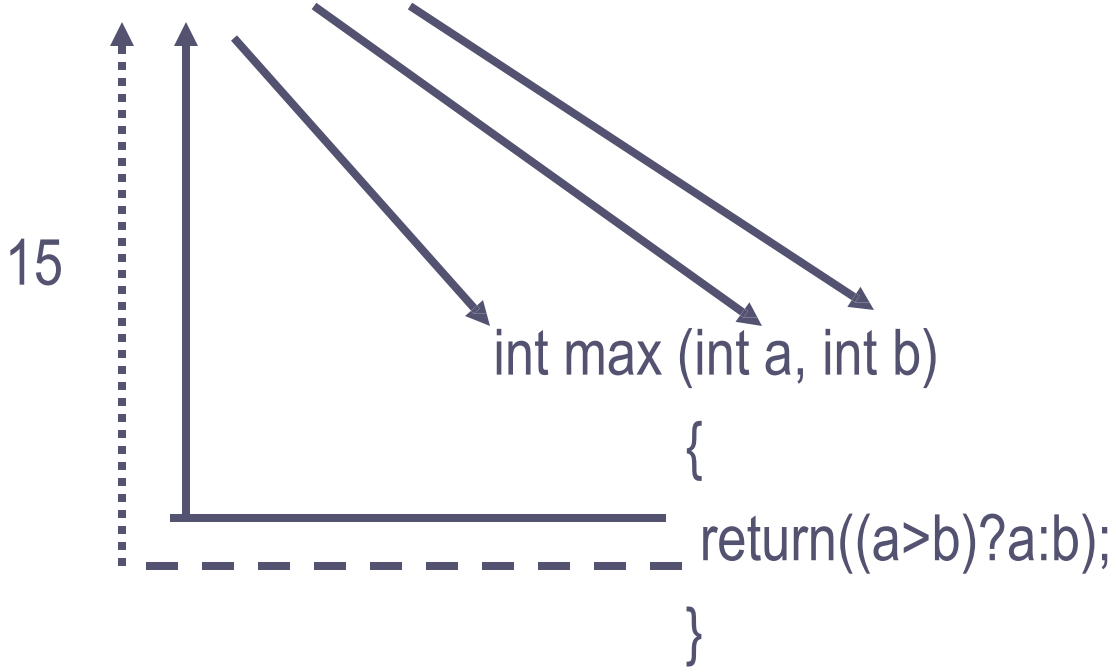
Παράδειγμα (2)

```
int max (int a, int b )
{
    int max;
    max = ( a > b) ? a : b;
    return (max);
}

int max( int a, int b)
{
    return ((a > b) ? a : b);
}
```

Μηχανισμός Κλήσης (1)

max_num = max (max (15, 13), 17);



Αναδρομή (Recursion) (1)

- Ορισμός συνάρτησης μέσω κλήσης του εαυτού της
- Μία τεχνική επίλυσης προβλημάτων
- Π.χ. εύρεση αθροίσματος 1+2+…+n

- Κλασσική λύση

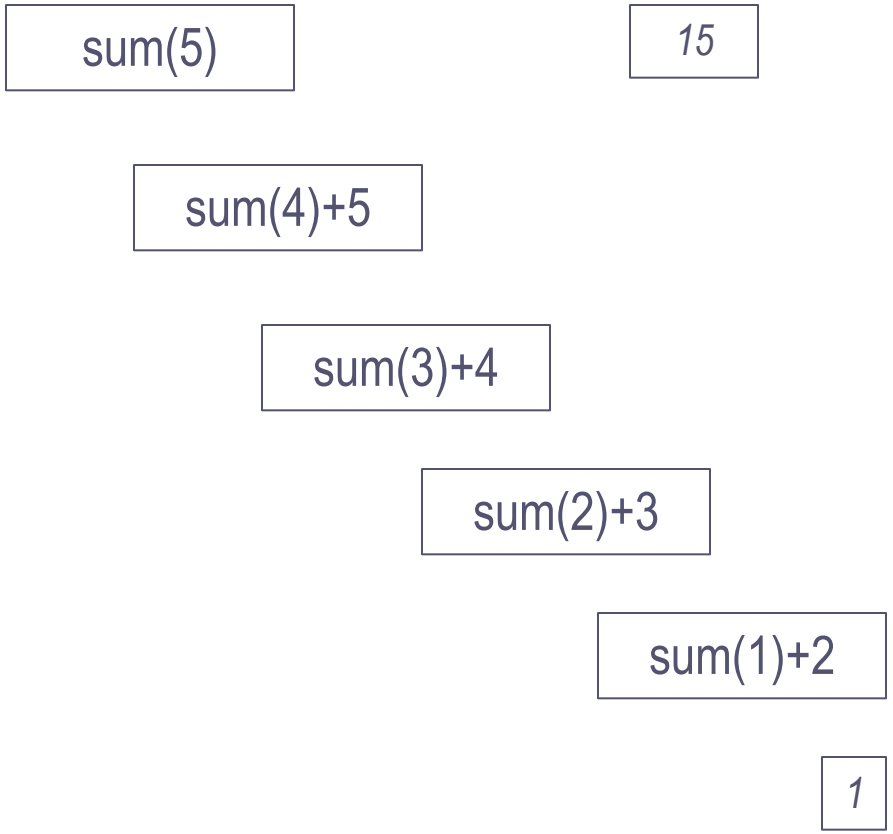
```
int sum (int n) {
s=0;
for (i=1; i<=n; i++)
  s=s+i;
return s;}
```

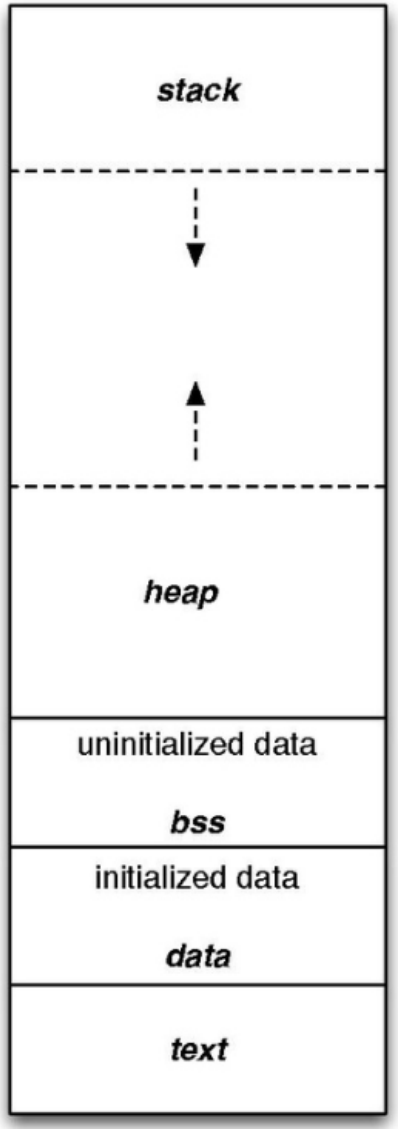
Αναδρομή (Recursion) (2)

- Αναδρομική λύση

  sum(n) = sum (n-1) + n

```
int sum (int n) {
if  (n<=1)
   return n;
else
   return (sum(n-1)+n);}
```





Memory Layout of a C program

https://commons.wikimedia.org/wiki/File:Program_memory_layout.pdf

This file is licensed under the Creative Commons Attribution-Share Alike 3.0 Unported license.

Η συνάρτηση main()

HelloWorld.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int i;

    printf("Hello world!\n");
    printf("argc=%d\n", argc);
    for(i=0;i<argc;i++)
        printf("argv[%d]=%s\n", i,argv[i]);
    return 0;
}
```

Η συνάρτηση από την οποία ξεκινά η εκτέλεση του προγράμματος.

argc: Ο αριθμός των παραμέτρων που περνάμε στην main όταν εκτελούμε το πρόγραμμα

argv: πίνακας δεικτών στα αλφαριθμητικά της γραμμής διαταγών (command line) με την οποία εκτελούμε το πρόγραμμα.

```
C:\Code\courses\I2P22-23\HelloWorld>helloworld "by kleanthis" Thramboulidis
Hello world!
argc=3
argv[0]=helloworld
argv[1]=by kleanthis
argv[2]=Thramboulidis
```

main(): Η συνάρτηση η οποία περιγράφει τη διεργασία που θα εκτελέσει ο υπολογιστής όταν τρέξουμε το πρόγραμμα.

Συνάρτηση - Ορισμός

Που είναι το interface και το implementation της συνάρτησης sqrt;

- είναι μία αυτόνομη μονάδα κώδικα σχεδιασμένη να επιτελεί συγκεκριμένο έργο.
- Π.χ. Έργο: Υπολογισμός τετραγωνικής ρίζας
  - Interface: **double sqrt(double x)**
  - Implementation: **?**

Όνομα ορίσματος

Τύπος ορίσματος

Τύπος επιστρεφόμενης τιμής

Όνομα συνάρτησης

Βασική Βιβλιοθήκη της C

- Ορίζει ένα σύνολο από συναρτήσεις που αναπαριστούν **βασικές διεργασίες**
- Η γνώση των συναρτήσεων αυτών αποτελεί βασική προϋπόθεση για την ανάπτυξη σύνθετων προγραμμάτων

Standard C library
https://www.csse.uwa.edu.au/programming/ansic-library.html
Standard C Library Functions Table, By Name - IBM
https://www.ibm.com/docs/en/i/7.3?topic=extensions-standard-c-library-functions-table-by-name

1. ANS X3.159-1989 πρότυπο
τον Δεκέμβριο του 1989 και δημοσιεύθηκε άνοιξη του 1990
2. ISO 9899-1990 πρότυπο
είναι σχεδόν αντίγραφο του ANSI. (ISO:International Organization for Standardizatio
3. ISO/IEC 899:1999, πρότυπο αντικατέστησε το 1990 ISO πρότυπο
. . .

Βασικές κατηγορίες συναρτήσεων της standard libray

- εισόδου εξόδου
- μαθηματικές
- διαχείρισης αλφαριθμητικών
- ταξινόμηση χαρακτήρων και μετατροπές
  - χαρακτήρων
- μετατροπής δεδομένων
- αναζήτησης και ταξινόμησης

- διαχείρισης αρχείων
- διαχείρισης μνήμης

Συναρτήσεις I/O της ΒΒ

- **Εισόδου**

```
int getchar(void);
int scanf(const char* format, ...);
```

- **Εξόδου**

```
int putchar(int c);
int puts(const char* s);
int    printf(const    char    *format,
όρισμα2, όρισμα3,…);
```

Συνάρτηση της BB

Η συνάρτηση printf

```
#include <stdio.h>  /* αρχείο επικεφαλίδας */
```

Το **πρωτότυπο** της (function prototype) είναι:

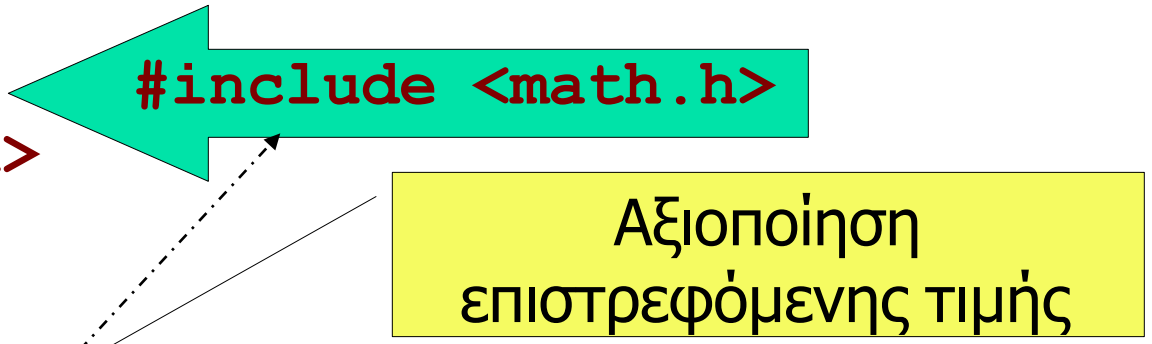
```
int printf(const char *format, όρισμα2,…);
```

- Το πρώτο όρισμα ορίζει σύμφωνα με δεδομένους κανόνες (δες εγχειρίδιο βασικής βιβλιοθήκης) τον αριθμό και τον τύπο των ορισμάτων που θα εξάγει η συνάρτηση στην κύρια έξοδο. Αυτά θα πρέπει να είναι σε συμφωνία με τα υπόλοιπα ορίσματα που ακολουθούν. Τις τιμές αυτών των ορισμάτων βγάζει η συνάρτηση στην έξοδο.

Χρήση συνάρτησης της BB -1

```
#include <stdio.h>
main()
{
int num1 = 2;
int num2 = 4;
int sum;
sum = 2 + 4;
printf("2 + 4 = %d\n",sum);
printf("%d + %d = %d\n",num1,num2,sum);
}
```

Χρήση συνάρτησης της ΒΒ -2




```
#include <stdio.h>
main()
{
double num1 = 16;
double result;
result = sqrt(num1);
printf("sqrt of %f is %f\n",num1,result);
printf("sqrt of %f is %f\n",num1, sqrt(num1) );
}
```